

OPTION B INSTITUTE FAMILY

M14 - KAFKA + STREAMING FOUNDATIONS

Standalone Zero-to-Job-Ready Data Engineer 2026

BEGINNER-FIRST MASTER REPAIR RC4

12 canonical lessons | full 16-stage teacher contract | protected attempt-first review

This learner book is rebuilt from the frozen canonical technical material. It is not a front-matter patch: every lesson is re-laid out into the shared institute teaching grammar while preserving the course competency, source route, guided work, independent proof and validation boundary.

This book is your teacher

Use the book in order. External videos and official docs are visual companions, not hidden prerequisites. Watch or read only the assigned scope, then close it before the independent task.

The 16-stage lesson loop

Why it matters -> What you learn -> Prerequisite -> Picture it -> New words -> Useful source -> Follow once -> Expected result -> Why it worked -> Beginner mistakes -> Recovery -> Independent task -> Validate -> Teach it back -> Protected review + changed retry -> Evidence/mastery + exact next route.

Attempt-first protection

Never open a protected lesson review before saving a genuine independent attempt. If repair is needed, compare only the weak idea, close the review, solve a changed retry, then validate again. A solution seen during review is repair evidence, not mastery.

What counts as mastery

A reproducible artifact or explanation, an independent validation control, preserved failure/recovery evidence, a changed transfer task completed without the worked answer open, and an own-words explanation. Percent-complete is not a substitute.

How to use your devices

Phone: assigned video/reference, reading, revision. Laptop/PC: every real command, query, file, workbook, code task, validation and evidence artifact. Install only what the current lesson asks for.

Contents / Index

This book is your teacher	2
Contents / Index	3
Prerequisite / tool map	4
Practice / review / Gate route	4
DE14-01 - Batch vs streaming	5
DE14-02 - Events, brokers and topics	7
DE14-03 - Producers and message keys	9
DE14-04 - Consumers and safe processing	11
DE14-05 - Partitions and ordering	13
DE14-06 - Offsets, group position and replay	15
DE14-07 - Consumer groups and scaling	17
DE14-08 - Delivery semantics and idempotency	19
DE14-09 - Serialization, schemas and evolution	21
DE14-10 - Python producer and consumer workflow	23
DE14-11 - Spark Structured Streaming + Kafka	25
DE14-12 - Lag, failures and streaming operations	27
Module mastery checklist + revision quick reference	29

Prerequisite / tool map

Understand Kafka/event streaming, offsets, replay, delivery semantics and Structured Streaming foundations.

Item	Rule
Entry prerequisite	Previous module/Gate evidence must be stable.
Primary device	PC/laptop for hands-on work; phone is support only.
External sources	Optional/supporting unless the lesson explicitly marks an official current reference.
Evidence	Module practical + fresh retest + saved evidence; no critical unresolved failure.

Practice / review / Gate route

Lesson teaching -> follow once -> independent attempt -> validate -> save evidence. Only after the genuine attempt may protected review open. If weak: targeted repair -> close review -> changed retry -> revalidate. Finish the module practical/retest route before any downstream Gate that depends on this module.

DE14-01 - Batch vs streaming

1. Why it matters

This matters because later Kafka + Streaming Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Build and explain the competency: Batch vs streaming.

3. Prerequisite / starting state

Starting state: no prior technical knowledge is assumed beyond the module Start Here instructions.

4. Picture it in your head

Batch processes bounded data at chosen intervals; streaming processes an ongoing event flow with progress and failure state. Streaming is not automatically faster or better: it adds operational complexity.

5. New words before we use them

Batch processes bounded data at chosen intervals; streaming processes an ongoing event flow with progress and failure state. Streaming is not automatically faster or better: it adds operational complexity. Choose it when latency and continuous reaction justify the cost.

6. Useful video / source

Source: Confluent Developer — free Apache Kafka courses

Assigned scope: Open the matching Kafka 101 lesson for this concept

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Classify three requirements as batch or streaming and justify latency/complexity trade-off. Classify three requirements as batch or streaming and justify latency/complexity trade-off.

8. Expected result / what you should see

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-01 - Practice and proof

11. If you get stuck

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

12. Small independent task

Design batch vs streaming choice for payment settlement and defend the latency/SLA boundary.

13. Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

14. Teach it back

Explain batch vs streaming in 60-90 seconds, including one failure mode and one validation. Fresh changed retry Changed clinic alerts: choose batch/streaming and define latency target.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery. Exact next route: DE14-02

DE14-02 - Events, brokers and topics

1. Why it matters

This matters because later Kafka + Streaming Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Build and explain the competency: Events, brokers and topics.

3. Prerequisite / starting state

Prerequisite: DE14-01 mastered with saved evidence.

4. Picture it in your head

An event is an immutable fact. Producers append events to topics managed by brokers.

5. New words before we use them

An event is an immutable fact. Producers append events to topics managed by brokers. A topic is split into partitions; each partition is an ordered append-only log with monotonically increasing offsets.

6. Useful video / source

Source: Confluent Developer — free Apache Kafka courses

Assigned scope: Open the matching Kafka 101 lesson for this concept

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Run the normal simulator; inspect topic partitions, events and offsets. Run the normal simulator; inspect topic partitions, events and offsets.

8. Expected result / what you should see

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-02 - Practice and proof

11. If you get stuck

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

12. Small independent task

Design topic/event envelope for parcel status events: key, value, timestamp and retention purpose.

13. Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

14. Teach it back

Explain events, brokers and topics in 60-90 seconds, including one failure mode and one validation. Fresh changed retry Changed IoT domain: define topic and immutable event fields.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery. Exact next route: DE14-03

DE14-03 - Producers and message keys

1. Why it matters

This matters because later Kafka + Streaming Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Build and explain the competency: Producers and message keys.

3. Prerequisite / starting state

Prerequisite: DE14-02 mastered with saved evidence.

4. Picture it in your head

A producer sends key/value events. The key is not just metadata: partitioning commonly uses it so related events land together.

5. New words before we use them

A producer sends key/value events. The key is not just metadata: partitioning commonly uses it so related events land together. Choose keys from ordering/affinity requirements, not randomly.

6. Useful video / source

Source: Confluent Developer — free Apache Kafka courses

Assigned scope: Open the matching Kafka 101 lesson for this concept

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Trace each event key to its partition and prove one key never jumps partitions. Trace each event key to its partition and prove one key never jumps partitions.

8. Expected result / what you should see

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-03 - Practice and proof

11. If you get stuck

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

12. Small independent task

Choose a message key for account transactions and explain ordering consequence.

13. Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

14. Teach it back

Explain producers and message keys in 60-90 seconds, including one failure mode and one validation. Fresh changed retry Changed shipment events: choose key so each shipment stays ordered.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery. Exact next route: DE14-04

DE14-04 - Consumers and safe processing

1. Why it matters

This matters because later Kafka + Streaming Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Build and explain the competency: Consumers and safe processing.

3. Prerequisite / starting state

Prerequisite: DE14-03 mastered with saved evidence.

4. Picture it in your head

A consumer polls records, performs work, and advances/commits progress. The dangerous moment is between side effect and offset commit: a crash can cause replay.

5. New words before we use them

A consumer polls records, performs work, and advances/commits progress. The dangerous moment is between side effect and offset commit: a crash can cause replay. Safe consumers therefore assume reprocessing can happen and make sinks idempotent or coordinate output/progress atomically.

6. Useful video / source

Source: Confluent Developer — free Apache Kafka courses

Assigned scope: Open the matching Kafka 101 lesson for this concept

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Trace the crash-before-offset-commit scenario and show which event replays. Trace the crash-before-offset-commit scenario and show which event replays.

8. Expected result / what you should see

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-04 - Practice and proof

11. If you get stuck

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

12. Small independent task

Design a consumer that writes to an external database safely when a crash can occur after write but before offset commit.

13. Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

14. Teach it back

Explain consumers and safe processing in 60-90 seconds, including one failure mode and one validation. Fresh changed retry Changed notification sink: handle crash/replay without duplicate sends.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery. Exact next route: DE14-05

DE14-05 - Partitions and ordering

1. Why it matters

This matters because later Kafka + Streaming Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Build and explain the competency: Partitions and ordering.

3. Prerequisite / starting state

Prerequisite: DE14-04 mastered with saved evidence.

4. Picture it in your head

Partitions are Kafka's unit of ordering and parallelism. Kafka guarantees order within a partition, not across an entire multi-partition topic.

5. New words before we use them

Partitions are Kafka's unit of ordering and parallelism. Kafka guarantees order within a partition, not across an entire multi-partition topic. If all events for one customer require order, a stable customer key can keep them together; more partitions increase parallelism but change capacity/operations.

6. Useful video / source

Source: Confluent Developer — free Apache Kafka courses

Assigned scope: Open the matching Kafka 101 lesson for this concept

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Verify offsets are monotonic inside each partition and explain why global order is not guaranteed.

Verify offsets are monotonic inside each partition and explain why global order is not guaranteed.

8. Expected result / what you should see

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-05 - Practice and proof

11. If you get stuck

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

12. Small independent task

Given 6 partitions and 9 consumers, predict active/idle consumers and ordering scope.

13. Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

14. Teach it back

Explain partitions and ordering in 60-90 seconds, including one failure mode and one validation. Fresh changed retry Changed topic has 4 partitions and 6 consumers: predict assignment capacity.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery. Exact next route: DE14-06

DE14-06 - Offsets, group position and replay

1. Why it matters

This matters because later Kafka + Streaming Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Build and explain the competency: Offsets, group position and replay.

3. Prerequisite / starting state

Prerequisite: DE14-05 mastered with saved evidence.

4. Picture it in your head

Offsets identify positions inside one partition. A consumer group commits positions so a restarted member can resume.

5. New words before we use them

Offsets identify positions inside one partition. A consumer group commits positions so a restarted member can resume. Resetting to earliest/latest changes what unread history is considered.

6. Useful video / source

Source: Confluent Developer — free Apache Kafka courses

Assigned scope: Open the matching Kafka 101 lesson for this concept

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Use committed_positions and replay list to reconstruct restart behavior. Use committed_positions and replay list to reconstruct restart behavior.

8. Expected result / what you should see

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-06 - Practice and proof

11. If you get stuck

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

12. Small independent task

Write a replay plan from a saved offset without duplicating already-published business effects.

13. Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

14. Teach it back

Explain offsets, group position and replay in 60-90 seconds, including one failure mode and one validation. Fresh changed retry Changed group starts from a different committed offset: reconstruct replay.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery. Exact next route: DE14-07

DE14-07 - Consumer groups and scaling

1. Why it matters

This matters because later Kafka + Streaming Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Build and explain the competency: Consumer groups and scaling.

3. Prerequisite / starting state

Prerequisite: DE14-06 mastered with saved evidence.

4. Picture it in your head

Consumers sharing a group divide partitions so one partition is handled by one group member at a time. Adding consumers beyond partition count creates idle members.

5. New words before we use them

Consumers sharing a group divide partitions so one partition is handled by one group member at a time. Adding consumers beyond partition count creates idle members. Membership changes can rebalance assignments, so processing should tolerate handoff/restart and not keep critical state only in one process memory.

6. Useful video / source

Source: Confluent Developer — free Apache Kafka courses

Assigned scope: Open the matching Kafka 101 lesson for this concept

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Inspect assignment of 3 partitions to 4 consumers and explain the idle member. Inspect assignment of 3 partitions to 4 consumers and explain the idle member.

8. Expected result / what you should see

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-07 - Practice and proof

11. If you get stuck

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

12. Small independent task

Explain what happens during a group rebalance and what local state must not be assumed durable.

13. Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

14. Teach it back

Explain consumer groups and scaling in 60-90 seconds, including one failure mode and one validation. Fresh changed retry Changed membership: one consumer dies; explain reassignment and safe state.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery. Exact next route: DE14-08

DE14-08 - Delivery semantics and idempotency

1. Why it matters

This matters because later Kafka + Streaming Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Build and explain the competency: Delivery semantics and idempotency.

3. Prerequisite / starting state

Prerequisite: DE14-07 mastered with saved evidence.

4. Picture it in your head

At-least-once processing can repeat work after failure; at-most-once can lose work; exactly-once claims are scoped. Kafka idempotent producers and transactions can make Kafka-to-Kafka processing strong, but transactions do not magically cover arbitrary external APIs/databases.

5. New words before we use them

At-least-once processing can repeat work after failure; at-most-once can lose work; exactly-once claims are scoped. Kafka idempotent producers and transactions can make Kafka-to-Kafka processing strong, but transactions do not magically cover arbitrary external APIs/databases. State the exact boundary of the guarantee.

6. Useful video / source

Source: Confluent Developer — free Apache Kafka courses

Assigned scope: Open the matching Kafka 101 lesson for this concept

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Compare raw processed+replayed operations with unique idempotent sink event IDs. Compare raw processed+replayed operations with unique idempotent sink event IDs.

8. Expected result / what you should see

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-08 - Practice and proof

11. If you get stuck

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

12. Small independent task

State precisely what “exactly once” does and does not guarantee for Kafka→REST API.

13. Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

14. Teach it back

Explain delivery semantics and idempotency in 60-90 seconds, including one failure mode and one validation. Fresh changed retry Changed external payment API: define dedup/idempotency boundary.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery. Exact next route: DE14-09

DE14-09 - Serialization, schemas and evolution

1. Why it matters

This matters because later Kafka + Streaming Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Build and explain the competency: Serialization, schemas and evolution.

3. Prerequisite / starting state

Prerequisite: DE14-08 mastered with saved evidence.

4. Picture it in your head

Serialization turns event objects into bytes. A schema makes field names/types/contracts explicit.

5. New words before we use them

Serialization turns event objects into bytes. A schema makes field names/types/contracts explicit. Evolution needs compatibility rules so old/new producers and consumers can coexist.

6. Useful video / source

Source: Confluent — How to use Data Contracts for Long-Term Schema Management

Assigned scope: 8:38–31:17: Avro/Protobuf, schema references, format changes, forward/backward/full compatibility

How to use it: Use this longer official class for serialization and compatibility thinking. Then complete the Teaching Book schema-evolution task with the video closed.

Note: This replaces the shorter registry-only visual because it directly covers the formats and evolution decisions the lesson needs.

[Open source](#)

7. Follow with me once

Compare schema v1/v2 and classify additive vs breaking change. Compare schema v1/v2 and classify additive vs breaking change.

8. Expected result / what you should see

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-09 - Practice and proof

11. If you get stuck

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

12. Small independent task

Classify four schema changes under backward/forward compatibility reasoning.

13. Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

14. Teach it back

Explain serialization, schemas and evolution in 60-90 seconds, including one failure mode and one validation. Fresh changed retry Changed schema adds nullable device_model then removes event_id: classify safety.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery. Exact next route: DE14-10

DE14-10 - Python producer and consumer workflow

1. Why it matters

This matters because later Kafka + Streaming Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Build and explain the competency: Python producer and consumer workflow.

3. Prerequisite / starting state

Prerequisite: DE14-09 mastered with saved evidence.

4. Picture it in your head

A Python Kafka workflow has configuration, producer callbacks/flush, consumer polling/error handling, explicit group ID, offset policy and controlled commits. Credentials belong in environment/config, not source.

5. New words before we use them

A Python Kafka workflow has configuration, producer callbacks/flush, consumer polling/error handling, explicit group ID, offset policy and controlled commits. Credentials belong in environment/config, not source. Save topic, partition, offset and event ID evidence so failures can be diagnosed.

6. Useful video / source

Source: DataTalksClub — real-time streaming workshop

Assigned scope: Producer + consumer sections

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Read producer.py/consumer.py line by line and map every config to behavior. Read producer.py/consumer.py line by line and map every config to behavior.

8. Expected result / what you should see

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-10 - Practice and proof

11. If you get stuck

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

12. Small independent task

Write your own Python producer/consumer skeleton without copying the reference files.

13. Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

14. Teach it back

Explain python producer and consumer workflow in 60-90 seconds, including one failure mode and one validation. Fresh changed retry Changed Python domain: produce/consume clinic events with explicit group and manual commit.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery. Exact next route: DE14-11

DE14-11 - Spark Structured Streaming + Kafka

1. Why it matters

This matters because later Kafka + Streaming Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Build and explain the competency: Spark Structured Streaming + Kafka.

3. Prerequisite / starting state

Prerequisite: DE14-10 mastered with saved evidence.

4. Picture it in your head

Spark Structured Streaming can use Kafka as a source/sink while applying DataFrame transformations incrementally. Kafka offsets identify source progress; Spark checkpoint state belongs to the streaming query.

5. New words before we use them

Spark Structured Streaming can use Kafka as a source/sink while applying DataFrame transformations incrementally. Kafka offsets identify source progress; Spark checkpoint state belongs to the streaming query. Do not delete a checkpoint to “fix” a job without understanding that you are changing its recovery identity.

6. Useful video / source

Source: DataTalksClub — real-time streaming workshop

Assigned scope: Stream-processing integration section; Teaching Book maps concept to Spark Structured Streaming

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Read Spark reference code and identify Kafka source options vs checkpoint location. Read Spark reference code and identify Kafka source options vs checkpoint location.

8. Expected result / what you should see

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-11 - Practice and proof

11. If you get stuck

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

12. Small independent task

Design Kafka→Spark transformation with stable checkpoint identity and rerun/restart evidence.

13. Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

14. Teach it back

Explain spark structured streaming + kafka in 60-90 seconds, including one failure mode and one validation. Fresh changed retry Changed Spark stream: explain consequences of changing/deleting checkpoint.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery. Exact next route: DE14-12

DE14-12 - Lag, failures and streaming operations

1. Why it matters

This matters because later Kafka + Streaming Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Build and explain the competency: Lag, failures and streaming operations.

3. Prerequisite / starting state

Prerequisite: DE14-11 mastered with saved evidence.

4. Picture it in your head

Consumer lag is the distance between the latest available offset and a group's committed/processed position. Rising lag can mean slow processing, insufficient parallelism, downstream failures or a stuck consumer.

5. New words before we use them

Consumer lag is the distance between the latest available offset and a group's committed/processed position. Rising lag can mean slow processing, insufficient parallelism, downstream failures or a stuck consumer. Diagnose lag together with throughput, errors, rebalance history and sink health; do not scale blindly.

6. Useful video / source

Source: DataTalksClub — real-time streaming workshop

Assigned scope: Failure/restart/operational portions + course incident task

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Use `lag_by_partition` to identify which partition should be investigated first. Use `lag_by_partition` to identify which partition should be investigated first.

8. Expected result / what you should see

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-12 - Practice and proof

11. If you get stuck

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

12. Small independent task

Diagnose rising lag with a five-step evidence-first incident sequence.

13. Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

14. Teach it back

Explain lag, failures and streaming operations in 60-90 seconds, including one failure mode and one validation. Fresh changed retry Changed incident: lag rises only on one partition; diagnose hot-key/downstream possibilities.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery. Exact next route: Proceed only when the M14 mastery checklist and downstream Gate route permit it.

Module mastery checklist + revision quick reference

- ☐ Every lesson has a saved genuine independent attempt.
- ☐ Every lesson has at least one independent validation/control appropriate to the tool.
- ☐ Protected reviews were opened only after attempts; any weak lesson has a changed retry after review was closed.
- ☐ You can teach back each lesson in your own words and name one realistic failure mode.
- ☐ Required module practical/retest evidence is saved and no critical defect is unresolved.
- ☐ D+1, D+4, D+10 and D+30 revision events are scheduled for weak or high-value competencies.

Revision cadence

D+1: fresh micro-task. D+4: mixed retrieval. D+10: changed/failure case. D+30: interview/project retrieval. Always attempt from memory before reopening a source.

Release boundary

This book is a static learner artifact. It does not prove live tool/cloud execution or a protected Gate pass. Runtime evidence must be recorded separately when the curriculum requires it.